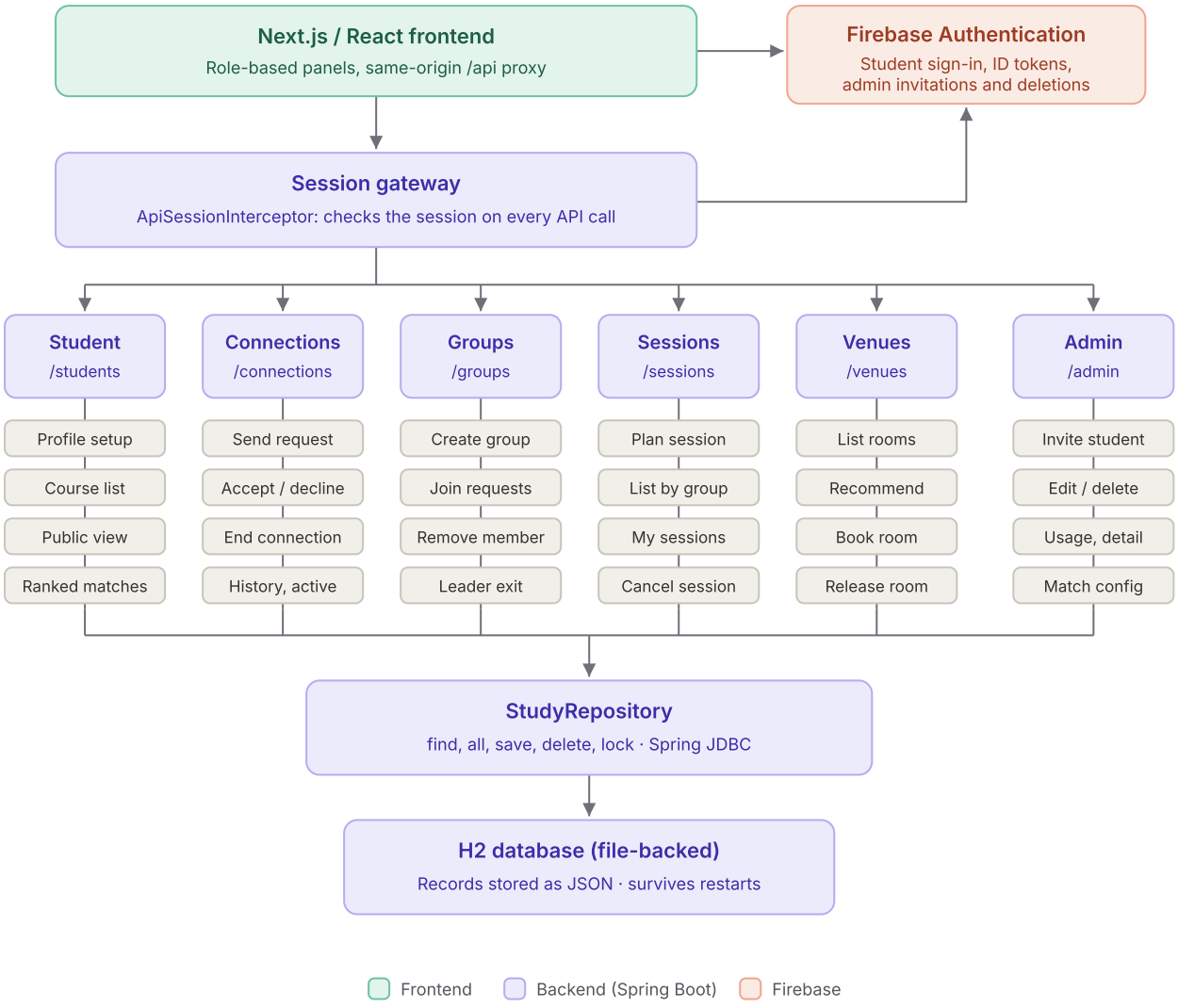
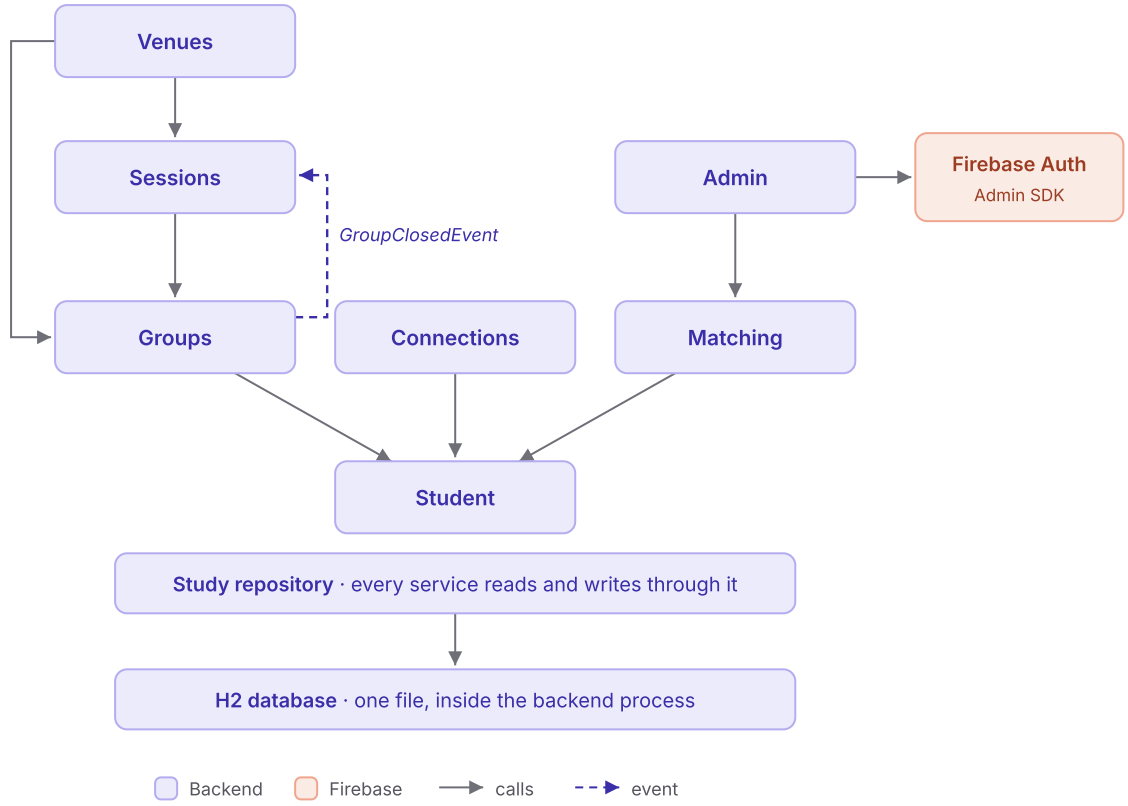
Study Buddy Matcher — Architecture Reference

1. System architecture



Students sign in with Firebase Authentication and the backend issues its own session; the gateway checks that session on every API call. Six modules sit behind it. Every service persists through `StudyRepository`, which stores each record as JSON in an H2 database file that runs inside the backend. Firebase holds no study data: it is used only for sign-in and for the admin's student invitations and deletions.

2. Module dependency graph



An arrow means “calls”. Connections, Groups and Matching call Student. Sessions calls Groups. Venues calls Sessions and Groups. Admin calls Matching (from its controller, for the config endpoints) and Firebase Authentication, to invite and delete students. The matches route lives in `StudentController`, which calls `MatchingService`; at service level, Matching depends on Student.

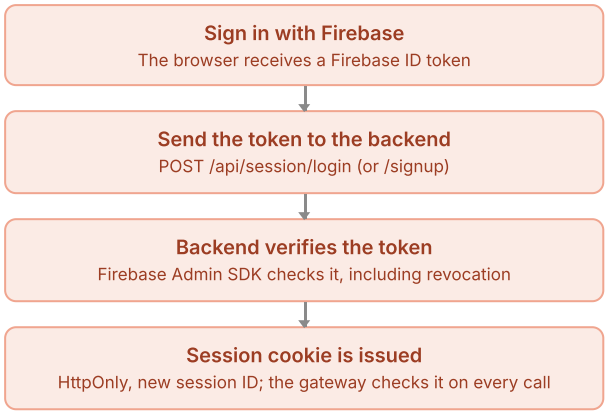
The dashed arrow is an event, not a call. When a group closes, `StudyGroupService` publishes `GroupClosedEvent` and `StudySessionService` removes that group’s sessions and room bookings, so Groups never has to know that Sessions exists.

> All arrows are plain Java method calls inside one Spring Boot process (constructor injection), not network calls. The event is a Spring application event, delivered in the same process and inside the same database transaction. REST only exists between the browser and the backend. All services also read and write through `StudyRepository`, which is why it is drawn once at the bottom.

> Admin is still the outlier: `AdminService` uses no other domain service. It reads and writes profiles, requests and groups straight through the repository. Deleting an account removes the Firebase identity first, then the profile, requests and memberships in one H2 transaction.

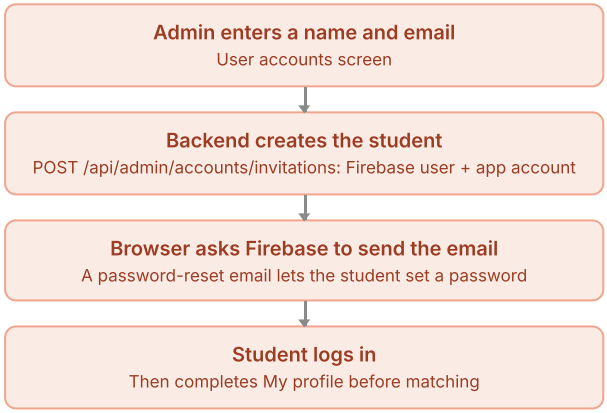
3. Process flows

Logging in



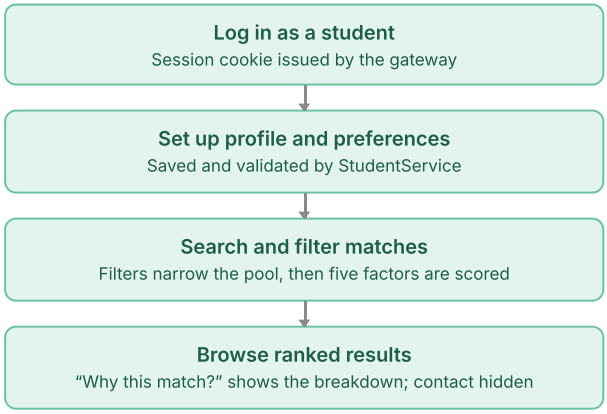
Students sign in through Firebase. The single administrator uses POST /api/session/admin-login with a username and password read from application.properties, which environment variables can override.

Inviting a student



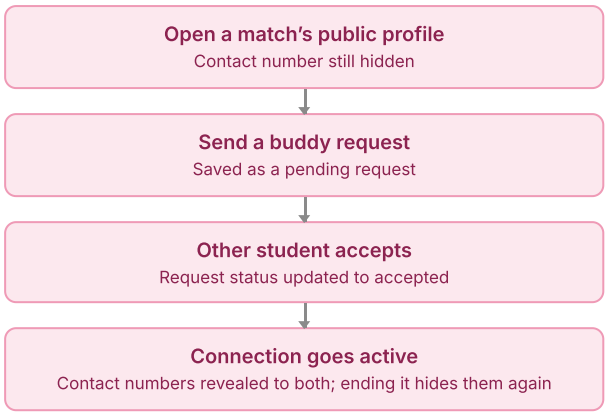
If saving the app account fails after Firebase has created the user, the backend deletes that Firebase user again. Students can also register themselves.

Finding a match



Results are ranked by the strategy the admin picked: Balanced (default), Availability first, Course first or Pareto fronts. The appendix lists the five factors and their weights.

Becoming study buddies



Forming a study group



Acceptance is re-checked when the leader decides, so two students can ask for the last seat and only one acceptance succeeds.

Planning a session and booking a room



Only the group leader (or an administrator) can plan or cancel a session. Any group member can request recommendations, book, or release a room. A room must seat the larger of the session size and the group's current members. When a group closes, `GroupClosedEvent` removes its sessions and bookings in the same transaction.

4. Module and API reference

Session and login `/api/session`

Method	Path	Function
POST	`/api/session/login`	Verify a Firebase ID token and start a session
POST	`/api/session/signup`	Verify the token, create a student account, start a session
POST	`/api/session/admin-login`	Administrator username and password login
GET	`/api/session`	Current logged-in account
DELETE	`/api/session`	Log out

Every other `/api` route needs an active session, and every change (non-GET) needs the header `X-StudyBuddy-Request: 1`.

Student `/api/students`

Method	Path	Function
GET	`/api/students`	List public profiles (contact hidden)
GET	`/api/students/courses`	Course catalogue
GET	`/api/students/{id}`	Viewer-aware public profile
POST	`/api/students`	Create or update profile, courses and preferences
GET	`/api/students/{id}/matches`	Ranked matches with score breakdown; filters for course, goal, mode, arrangement, day/time, overlap

The matching engine lives here (no separate controller): this route calls MatchingService. StudentService itself depends on StudyRepository only.

Connections `/api/connections`

Method	Path	Function
POST	`/api/connections/requests`	Send `{ receiverId, message }`; the sender comes from the session
POST	`/api/connections/requests/{id}/status`	Accept, decline or end (`status=ACCEPTED` , `DECLINED` or `ENDED`)
GET	`/api/connections/students/{id}`	Request history (inbox and sent)
GET	`/api/connections/students/{id}/active`	Active connections

Depends on StudentService (validation, contact reveal) and StudyRepository.

Groups `/api/groups`

Method	Path	Function
GET	`/api/groups`	List groups
GET	`/api/groups/join-requests`	Own and manageable join requests
POST	`/api/groups`	Create a group
PUT	`/api/groups/{id}`	Update group details
POST	`/api/groups/{id}/join-requests`	Student asks to join
POST	`/api/groups/join-requests/{id}/decision`	Leader accepts or declines
POST	`/api/groups/{id}/remove-member`	Remove a member, or leave as an ordinary member
POST	`/api/groups/{id}/transfer-leadership`	Transfer leadership and stay
POST	`/api/groups/{id}/close`	Close the group
POST	`/api/groups/{id}/leader-quits`	Leader exit: transfer and leave, or close and leave

Depends on StudentService and StudyRepository. Publishes `GroupClosedEvent` when a group closes.

Sessions `/api/sessions`

Method	Path	Function
POST	`/api/sessions`	Plan a study session for a group (leader)
GET	`/api/sessions?groupId=`	Sessions of one group, sorted by date
GET	`/api/sessions/mine`	Sessions across the caller's active groups
POST	`/api/sessions/{id}/cancel`	Cancel a session and its room booking (leader)

Depends on StudyGroupService and StudyRepository. Listens for `GroupClosedEvent` and deletes that group's sessions and bookings.

Venues `/api/venues`

Method	Path	Function
GET	`/api/venues/rooms`	All rooms
GET	`/api/venues/recommendations?sessionId=`	Rooms that fit the session, smallest first; optional building, whiteboard, projector, power filters
POST	`/api/venues/bookings?sessionId=&roomId=`	Book a room for a session
POST	`/api/venues/bookings/release?sessionId=`	Release the room booking

Depends on StudySessionService, StudyGroupService and StudyRepository. The 18 rooms are fictional demo data.

Admin `/api/admin`

Method	Path	Function
GET	`/api/admin/accounts`	Accounts with usage (profile, connections, groups, pending, last sign-in)
POST	`/api/admin/accounts`	Update an existing account (name, status)
POST	`/api/admin/accounts/invitations`	Create a Firebase-backed student from a name and email
DELETE	`/api/admin/accounts/{id}`	Delete the Firebase identity, then the account and related records
GET	`/api/admin/accounts/{id}/detail`	An account's buddies and groups
GET	`/api/admin/matching-config`	Current matching weights, strategy and result limit
PUT	`/api/admin/matching-config`	Update weights, strategy and result limit

AdminService depends on StudyRepository and Firebase (Admin SDK). The controller uses MatchingService for the config endpoints only.

Errors

All errors come back as `{ "error": "Readable message" }`: 400 for validation, 401 with no session, 403 for permission failures, 404 for missing records or unknown routes, 405 for a wrong method, and 409 for conflicting actions such as a double booking.

5. Appendix

Database tables (H2)

The database is one file, `apps/backend/data/studybuddy.mv.db`, created from `schema.sql`. It has two tables:

Table	Holds
`aggregates(kind, id, payload)`	Every record. `kind` is the Java class name, `id` the record's id, and `payload` the whole object as JSON.
`mutation_lock(id)`	One row that every business write locks first, so writes happen one at a time.

Kinds stored in `aggregates`:

Kind	Holds
`UserAccount`	Login account, role, status
`StudentProfile`	Name, courses, availability, preferences
`Course`	Course code and name
`BuddyRequest`	Buddy request and its status
`StudyGroup`	Group details, leader, members
`MembershipRequest`	Request to join a group
`StudySession`	Scheduled study sessions
`Room` / `RoomBooking`	Venues and bookings
`MatchingProperties`	Admin matching weights and strategy (id: `current`)
`SeedMarker`	Marks that demo data has been loaded

How writes stay consistent

Service methods that change data are marked `@Transactional`. Each one first calls `repository.lock()`, which runs `SELECT id FROM mutation_lock WHERE id = 1 FOR UPDATE`, so two writes at the same moment go one after the other. If the method throws an error, H2 rolls the whole change back. Reads build fresh objects from JSON, so a failed operation cannot leave half-changed objects behind.

Matching engine at a glance

Factor	Default weight	Full marks when
Course	40	Peer's preferred course is the search course (half if only enrolled)
Availability	25	Share of your free minutes the peer is also free (weeks are stored as 10,080 one-minute slots)
Meeting mode	15	Same mode, or either student chose "Either"
Study goal	15	Same primary goal
Group size	5	Same size and compatible arrangement; less as sizes differ

Weights are normalised to 100 and a weight of 0 turns a factor off. Strategies: Balanced (by score), Availability first, Course first, Pareto fronts. Defaults live under `app.matching.*` in application.properties until an admin saves new ones.

What happens when the API is called (example: sending a buddy request)

1. **Student clicks the button in the browser.**
   React builds a JSON object like `{ receiverId, message }`. The sender is not in the JSON; the backend takes it from the session.
2. **The browser sends an HTTP request.**
   A POST to `/api/connections/requests` with that JSON as the body and the header `X-StudyBuddy-Request: 1`. Next.js forwards `/api` to the backend.
3. **The gateway checks it.**
   `ApiSessionInterceptor` confirms the header and the active session.
4. **Spring Boot routes it.**
   `@PostMapping("/requests")` on `ConnectionController` maps the URL to a Java method.
5. **The backend does the work.**
   `ConnectionController` → `ConnectionService` (validates via `StudentService`) → `StudyRepository`, under the lock and inside one transaction.
6. **The backend responds.**
   The saved request, or an error such as `{ "error": "Student profile not found." }`.
7. **React updates the screen.**
   The frontend never touches the database or business rules directly.

Moving boxes in Inkscape

1. **Select the whole box.**
   Rectangle and text are not grouped, so drag a selection rectangle around the box.
2. **Group it**
   with Ctrl+G (Cmd+G on Mac) so it moves as one object.
3. **Drag or nudge**
   with the mouse or arrow keys; Object > Transform (Shift+Ctrl+M) for exact values.
4. **Reconnect arrows by hand.**
   Lines are not attached to boxes; use the Node tool (N) to drag their endpoints.